

Kuromaku — Deployment Guide

A phased, checkpointed recon/vulnerability-scanning pipeline built on Docker worker containers and an MCP orchestrator, controllable from LM Studio via a local LLM.

Scope: domain-based recon -> network -> webscan -> xss -> report pipeline, plus a standalone IP/CIDR scanning mode for DMZ exposure audits.

Authorization: only scan targets you have explicit written authorization to test. Every scan tool in this stack prints a reminder of this; it is not optional.

1. Prerequisites

1.1 Install Docker Desktop

Pick your platform:

- **Windows:** <https://www.docker.com/products/docker-desktop/> — requires WSL2 (the installer will prompt to enable it if missing). After install, open Docker Desktop once and let it finish initializing the WSL2 backend.
- **macOS:** <https://www.docker.com/products/docker-desktop/> — choose Apple Silicon or Intel build matching your Mac.
- **Linux:** install Docker Engine via your distro's package manager (apt install docker.io / dnf install docker / etc.), or Docker Desktop for Linux if you prefer the GUI. Add your user to the docker group so you don't need sudo for every command:

```
sudo usermod -aG docker $USER
# log out/in for this to take effect
```

Resource allocation (Docker Desktop -> Settings -> Resources): allocate memory based on your hardware. This stack has been validated at **8GB total** on an 8GB Mac Mini (tight but stable with the low resource profile) and runs comfortably with **8GB dedicated to the webscan container alone** on higher-spec hardware (high profile). See §5.3 for tuning.

File sharing (Mac/Windows only): ensure the directory you'll clone/extract this project into is under a shared path (Docker Desktop -> Settings -> Resources -> File Sharing). By default Users/%USERPROFILE% is shared.

1.2 Install LM Studio

Download from <https://lmstudio.ai> — available for Windows, macOS, and Linux.

1. Install and launch LM Studio.
 2. In the model search/download tab, search for and download a local model. **Qwen3.5-9B** (or similar qwen3.5-9b variant) is the model this stack was developed and validated against — it's small enough to run alongside Docker containers on consumer hardware while still handling tool-call syntax reliably. Any tool-calling-capable local model should work, but smaller models ($\leq 9B$) are more prone to **hallucinating interpretations of tool results** — always cross-check anything the model says against kuromaku_status output or the raw .checkpoint file / progress logs directly if something looks off.
 3. Load the model and confirm it shows "tool use" / "function calling" support in LM Studio's model card.
-

2. Project Directory Structure

The zipped project (excluding projects/, which holds per-engagement scan data and shouldn't be shared) should look like this:

```
bugbounty-workspace/
├── workers/
│   ├── recon/
│   │   ├── Dockerfile
│   │   └── phase-recon.sh
│   ├── network/
│   │   ├── Dockerfile
│   │   └── phase-network.sh
│   ├── webscan/
│   │   ├── Dockerfile
│   │   └── phase-webscan.sh
│   ├── xss/
│   │   ├── Dockerfile
│   │   └── phase-xss.sh
│   ├── report/
│   │   ├── Dockerfile
│   │   ├── phase-report.py
│   │   └── phase-report.sh
│   └── orchestrator/
│       ├── Dockerfile
│       ├── package.json
│       └── src/
│           ├── index.js
│           ├── orchestrator.js
│           └── checkpoint.js
└── projects/                                ← created at runtime, NOT included in
the zip
    ├── <project-name>/
    │   └── scans/
    │       ├── .checkpoint
    │       ├── *-uniqdomains.log
    │       ├── *-naabu.log
    │       ├── *-nucleiAlerts.log
    │       ├── *-dirsearch.log
    │       ├── *-dalfox.log
    │       ├── *-report.docx
    │       └── kuromaku_progress.log
```

After extracting the zip, your buddy should have everything under `workers/` populated. The `projects/` directory will be created automatically the first time a scan runs (or can be pre-created empty).

3. Build the Worker Images

Open a terminal/PowerShell in the extracted `bugbounty-workspace/workers/` directory. Build each of the six images — order doesn't matter, but doing the orchestrator last is convenient since it's the one you'll iterate on if something needs tuning.

```
cd workers/recon      && docker build -t kuromaku-recon .      && cd ..
cd workers/network    && docker build -t kuromaku-network .    && cd ..
cd workers/webscan    && docker build -t kuromaku-webscan .    && cd ..
cd workers/xss        && docker build -t kuromaku-xss .        && cd ..
cd workers/report     && docker build -t kuromaku-report .     && cd ..
cd workers/orchestrator && docker build -t kuromaku-orchestrator .
&& cd ..
```

Build times: recon, network, and xss are quick (Go binaries on Alpine, a few minutes each). webscan is the largest/slowest — it bakes in the full nuclei template set and nmap's vulners script, expect 5-15 minutes depending on connection speed. report and orchestrator are fast (Python/Node on Alpine).

Verify all six images exist:

```
docker images | grep kuromaku
```

You should see: `kuromaku-recon`, `kuromaku-network`, `kuromaku-webscan`, `kuromaku-xss`, `kuromaku-report`, `kuromaku-orchestrator`.

4. Run the Orchestrator

This is the one container that stays running persistently — it's the MCP server LM Studio talks to, and it spawns the other five worker images on-demand via the Docker socket.

4.1 The `HOST_PROJECTS_DIR` Requirement — Read This First

The orchestrator runs inside its own container but spawns *sibling* containers via the mounted Docker socket. Docker-from-Docker `-v` mounts are resolved by the **host** Docker daemon, not by paths inside the orchestrator container. This means:

- `-v <host-path>:/workspace/projects` mounts your projects directory into the orchestrator (so it can read/write checkpoints and results).
- `HOST_PROJECTS_DIR=<host-path>` (the **same** host path, passed as an env var) tells the orchestrator what that path is *on the host*, so when it spawns a worker container it can construct a correct `-v <host-path>/<project>/scans:/workspace/scans` for the worker.

These two values must match. Get this wrong and worker containers will fail to spawn with a “mounts denied” / “path is not shared from host” error.

4.2 Platform-Specific Run Commands

Choose a host directory for projects/ — e.g. ~/bugbounty-workspace/projects (Mac/Linux) or C:\Users\
<you>\Documents\bugbounty-workspace\projects (Windows). Create it if it doesn't exist.

macOS / Linux (bash/zsh)

```
export HOST_PROJECTS_DIR="$HOME/bugbounty-workspace/projects"
mkdir -p "$HOST_PROJECTS_DIR"

docker run -d --name kuromaku-orchestrator --restart unless-stopped \
    -e MCP_HTTP=true \
    -e MCP_HTTP_PORT=3002 \
    -e RESOURCE_PROFILE=low \
    -e HOST_PROJECTS_DIR="$HOST_PROJECTS_DIR" \
    -p 3002:3002 \
    -v /var/run/docker.sock:/var/run/docker.sock \
    -v "$HOST_PROJECTS_DIR:/workspace/projects" \
    kuromaku-orchestrator
```

Windows (PowerShell)

```
$HostProjectsDir = "C:\Users\$env:USERNAME\Documents\bugbounty-
workspace\projects"
New-Item -ItemType Directory -Force -Path $HostProjectsDir | Out-
Null

docker run -d --name kuromaku-orchestrator --restart unless-stopped \
    -e MCP_HTTP=true \
    -e MCP_HTTP_PORT=3002 \
    -e RESOURCE_PROFILE=low \
    -e HOST_PROJECTS_DIR=$HostProjectsDir \
    -p 3002:3002 \
    -v //var/run/docker.sock:/var/run/docker.sock \
    -v "${HostProjectsDir}:/workspace/projects" \
    kuromaku-orchestrator
```

If //var/run/docker.sock doesn't work on your Docker Desktop version, try /var/run/docker.sock (single slash) — Docker Desktop's path handling for the socket varies slightly by version.

4.3 Verify It's Running

```
curl http://localhost:3002/health
```

Expected: {"status":"running","version":"2.0.0"}

```
docker exec kuromaku-orchestrator env | grep HOST_PROJECTS_DIR
```

Confirm this matches the path you used in the -v mount exactly.

5. Configure LM Studio

5.1 Edit mcp.json

Location: - **macOS**: ~/Library/Application Support/LM Studio/mcp.json
- **Windows**: %APPDATA%\LM Studio\mcp.json - **Linux**: ~/.config/LM Studio/mcp.json

Add (or replace existing) entry:

```
{
  "mcpServers": {
    "kuromaku": {
      "type": "http",
      "url": "http://localhost:3002/mcp"
    }
  }
}
```

Reload MCP servers in LM Studio (usually a refresh icon in the MCP/plugins panel, or restart LM Studio). Confirm the following tools appear:

- kuromaku_run
- kuromaku_status
- kuromaku_resume
- kuromaku_pause
- kuromaku_stop
- kuromaku_results
- kuromaku_ip_scan
- kuromaku_new_project
- kuromaku_list_projects

5.2 First Smoke Test

In a new LM Studio chat with the model loaded:

List my bug bounty projects

This is read-only (kuromaku_list_projects) — confirms the MCP connection and the projects-directory mount work, without spawning any containers.

5.3 Resource Profiles

RESOURCE_PROFILE (set as an env var on the orchestrator container) selects a memory/concurrency preset applied to every spawned worker:

Profile	webscan memory	nuclei bulk- size / concurrency / rate-limit	Suitable for
low	3g	5 / 5 / 25	~8GB total system RAM, shared with other apps
high	8g	20 / 20 / 80	8GB+ dedicated to the webscan container

To change profiles, stop/remove and recreate the orchestrator container with a different `-e RESOURCE_PROFILE=...` value (\$4.2). This affects all *subsequent* worker spawns — an in-progress scan's current phase keeps its original settings until that phase finishes or is stopped/resumed.

If you need something between low and high (e.g. a medium profile), edit the `PROFILES` table in `workers/orchestrator/src/orchestrator.js` and rebuild the orchestrator image (`docker build --no-cache -t kuromaku-orchestrator .`).

6. Usage

6.1 Domain-Based Recon Pipeline

Start a Kuromaku scan on `example.com`, project name `"example-recon"`

This runs all five phases sequentially, auto-advancing: **recon -> network -> webscan -> xss -> report**. Each phase's output is checkpointed; if a phase fails, the pipeline stops and is resumable.

Optional parameters the model can pass: `- skip_nuclei: true` — webscan still runs `nmap + dirsearch`, skips the (slowest) nuclei template scan. `- skip_xss: true` — skip the XSS-probing phase entirely. `- threads: <n>` — concurrency for recon/dirsearch (default 10). `- llm_url / llm_model` — override the report phase's executive-summary LLM endpoint (defaults to `http://host.docker.internal:1234/v1`, i.e. LM Studio's own OpenAI-compatible API — the report phase can ask the *same* local model running in LM Studio to write its executive summary).

6.2 Monitoring

Check Kuromaku status for `"example-recon"`

For ground truth (bypassing any LLM interpretation), read the checkpoint and progress log directly:

```
cat <projects-dir>/example-recon/scans/.checkpoint
tail -f <projects-dir>/example-recon/scans/kuromaku_progress.log
```

6.3 Pause / Resume / Stop

- **Pause** — let the current phase finish, then stop before the next:

Pause Kuromaku scan for `"example-recon"` after webscan

- **Resume** — continue from the first pending/failed phase (clears any pause):

Resume Kuromaku scan for `"example-recon"`

- **Stop** — kill the currently-running phase's container immediately (marks it failed/resumable):

Stop Kuromaku scan for `"example-recon"`

6.4 IP/CIDR Scanning (DMZ Exposure Audits)

For scanning an IP range rather than a domain — e.g. “what’s exposed on our DMZ beyond what’s approved”:

Scan 203.0.113.0/24 for open ports, project name "dmz-audit"

This runs a simplified 2-phase pipeline: **network** (naabu port sweep) - > **webscan** (nmap service/version detection + vulners script). No hostname-dependent steps (recon, dirsearch, nuclei, xss) run for IP-only targets. Same pause/resume/stop/status tools apply.

Accepts a single IP (203.0.113.5/32), a CIDR (203.0.113.0/24), or a path to a file with one CIDR/IP per line.

7. Troubleshooting

“mounts denied” / “path is not shared from the host”
HOST_PROJECTS_DIR doesn’t match the host-side path used in the -v mount for the orchestrator, or contains a typo/wrong drive letter.
Recheck §4.1/4.2.

Worker container never appears in docker ps, orchestrator logs show an error

```
docker logs kuromaku-orchestrator --tail 50
```

Errors here show the exact docker run command that failed and Docker’s response — almost always a path or env-var mismatch.

LLM says a project “doesn’t exist” or “scan failed” but the checkpoint looks fine
Smaller local models occasionally narrate tool results inaccurately. Always verify against .checkpoint / kuromaku_progress.log directly before trusting a status summary.

A running phase seems stuck forever
The orchestrator auto-detects stale phases (container no longer running, started_at > 30 minutes ago) and marks them failed on the next status/resume call, making them resumable.

Nuclei batch is taking a very long time
This is expected and proportional to (templates × hosts) / rate-limit. A large domain count with low profile’s rate-limit=25 can mean many hours per batch (3 batches total). See §5.3 to increase the rate limit if you have the spare memory/CPU headroom — watch the Errors count in kuromaku_progress.log; a disproportionate spike usually means the target’s WAF is rate-limiting you back, not a local resource issue.

8. What’s Inside Each Worker (Reference)

Worker	Tools	Input	Output
recon	subfinder, amass, gau, katana, httpx	domain	*_
			uniqdomains.log,
			*_
			domains_only.log, *_gau-dirty.log

network	dnsx, naabu	recon output (or --cidr)	*-resolved.log, *-unique-ips.log, *-naabu.log
webscan	nuclei, nmap+vulners, dirsearch	recon+network output (or -- cidr, nmap- only)	*- nucleiAlerts.log, *-dirsearch.log, *- nmapvulners.log
xss	qsreplace, httpx (-mr reflection check), dalfox	recon's gau- dirty.log	*-dalfox.log
report	python-docx + optional LM Studio call	all of the above	*-report.docx

9. Security Reminder

This stack will actively send traffic to whatever target you point it at. Before sharing with anyone:

- Confirm they understand every scan requires **written authorization** for the target.
- The --skip-nuclei / low profile defaults are deliberately conservative — encourage starting there on any new target before scaling up rate limits.
- projects/ contains scan results (subdomains, open ports, vulnerability findings, generated reports) for real engagements — this is why it's excluded from the shared zip. Treat its contents as sensitive.